

HUEY_P_ClaudeCentric Trading System: Master Technical Specification

****Version:**** 1.0.0

****Release Date:**** June 2025

****Document Status:**** Final

****Prepared by:**** Lead System Architect

****Classification:**** Internal Use, Definitive Source of Truth

Table of Contents

1. [Title Page](#1-title-page)
2. [Executive Summary / Overview](#2-executive-summary--overview)
3. [Goals and Objectives](#3-goals-and-objectives)
4. [Background and Context](#4-background-and-context)
5. [Definitions and Acronyms](#5-definitions-and-acronyms)
6. [System Architecture](#6-system-architecture)
 - 6.1 [High-Level Architecture Diagram](#61-high-level-architecture-diagram)
 - 6.2 [Core Architectural Principles](#62-core-architectural-principles)
 - 6.3 [Technology Stack](#63-technology-stack)
 - 6.4 [Component Descriptions](#64-component-descriptions)
 - 6.5 [Python Backend Layer](#65-python-backend-layer)
 - 6.6 [Bridge Layer](#66-bridge-layer)
 - 6.7 [MT4 Terminal Layer](#67-mt4-terminal-layer)
 - 6.8 [Data Layer](#68-data-layer)
 - 6.9 [PowerShell Management Layer](#69-powershell-management-layer)
 - 6.10 [Excel Dashboard Layer](#610-excel-dashboard-layer)
 - 6.11 [External Dependencies](#611-external-dependencies)
7. [System Behavior and Workflows](#7-system-behavior-and-workflows)
 - 7.1 [Data Flow Diagram: Signal to Execution](#71-data-flow-diagram-signal-to-execution)
 - 7.2 [Signal Generation Workflow](#72-signal-generation-workflow)
 - 7.3 [Error Handling Workflow](#73-error-handling-workflow)
 - 7.4 [State Transitions](#74-state-transitions)
8. [Functional Requirements](#8-functional-requirements)
9. [Non-Functional Requirements](#9-non-functional-requirements)
 - 9.1 [Performance Requirements](#91-performance-requirements)
 - 9.2 [Reliability and Availability](#92-reliability-and-availability)
 - 9.3 [Security Requirements](#93-security-requirements)
 - 9.4 [Scalability Requirements](#94-scalability-requirements)
 - 9.5 [Usability Requirements](#95-usability-requirements)
10. [Data Models](#10-data-models)
 - 10.1 [Entity Relationship Diagram](#101-entity-relationship-diagram)
 - 10.2 [Core Data Models](#102-core-data-models)

11. [API Specifications](#11-api-specifications)
 - 11.1 [Communication Bridge API (SocketBridge.dll)](#111-communication-bridge-api-socketbridgedll)
 - 11.2 [Signal Message Format (Socket/Named Pipe)](#112-signal-message-format-socketnamed-pipe)
 - 11.3 [Error and Status Code Definitions](#113-error-and-status-code-definitions)
 - 11.4 [REST API Endpoints](#114-rest-api-endpoints)
 - 11.5 [WebSocket API](#115-websocket-api)
 - 11.6 [Internal Communication Protocol](#116-internal-communication-protocol)
12. [Configuration Management](#12-configuration-management)
 - 12.1 [Configuration Hierarchy](#121-configuration-hierarchy)
 - 12.2 [File Formats](#122-file-formats)
 - 12.3 [Core Configuration Files](#123-core-configuration-files)
 - 12.4 [Environment-Specific Configuration](#124-environment-specific-configuration)
 - 12.5 [Pair-Specific Configuration](#125-pair-specific-configuration)
 - 12.6 [Configuration Loading Process](#126-configuration-loading-process)
13. [System Dependency and Validation](#13-system-dependency-and-validation)
 - 13.1 [Change Impact Mapping](#131-change-impact-mapping)
 - 13.2 [Validation Rules](#132-validation-rules)
14. [Test Plan and Acceptance Criteria](#14-test-plan-and-acceptance-criteria)
 - 14.1 [Testing Pyramid](#141-testing-pyramid)
 - 14.2 [Unit Testing](#142-unit-testing)
 - 14.3 [Integration Testing](#143-integration-testing)
 - 14.4 [End-to-End (E2E) Testing](#144-end-to-end-e2e-testing)
 - 14.5 [Backtesting](#145-backtesting)
 - 14.6 [Tools and Frameworks](#146-tools-and-frameworks)
 - 14.7 [Test Execution Policy](#147-test-execution-policy)
 - 14.8 [Acceptance Test Cases](#148-acceptance-test-cases)
15. [Deployment Plan and Environments](#15-deployment-plan-and-environments)
 - 15.1 [Prerequisites](#151-prerequisites)
 - 15.2 [Environment Architecture](#152-environment-architecture)
 - 15.3 [First-Time Deployment Procedure](#153-first-time-deployment-procedure)
 - 15.4 [Routine Operations](#154-routine-operations)
 - 15.5 [Rollback Plan](#155-rollback-plan)
16. [Monitoring and Logging Strategy](#16-monitoring-and-logging-strategy)
 - 16.1 [Monitoring Architecture](#161-monitoring-architecture)
 - 16.2 [Key Metrics](#162-key-metrics)
 - 16.3 [Logging Configuration](#163-logging-configuration)
 - 16.4 [Alert Configuration](#164-alert-configuration)
17. [Security Considerations](#17-security-considerations)
 - 17.1 [Authentication and Authorization](#171-authentication-and-authorization)
 - 17.2 [Data Security](#172-data-security)
 - 17.3 [Network Security](#173-network-security)
 - 17.4 [Security Monitoring](#174-security-monitoring)

- 17.5 [Compliance and Audit](#175-compliance-and-audit)
- 18. [Backup and Recovery](#18-backup-and-recovery)
 - 18.1 [Backup Procedure](#181-backup-procedure)
 - 18.2 [Disaster Recovery Procedure](#182-disaster-recovery-procedure)
 - 18.3 [Recovery Time Objective (RTO)](#183-recovery-time-objective-rto)
- 19. [Coding Standards](#19-coding-standards)
 - 19.1 [Core Philosophy](#191-core-philosophy)
 - 19.2 [Git Commit Messages](#192-git-commit-messages)
 - 19.3 [MQL4 Style](#193-mql4-style)
 - 19.4 [Python Style](#194-python-style)
 - 19.5 [C++ Style](#195-c-style)
 - 19.6 [PowerShell Style](#196-powershell-style)
 - 19.7 [Configuration File Style](#197-configuration-file-style)
- 20. [Assumptions and Constraints](#20-assumptions-and-constraints)
- 21. [Open Questions and Known Risks](#21-open-questions-and-known-risks)
- 22. [Glossary and External References](#22-glossary-and-external-references)
- 23. [File Inventory and Dependencies](#23-file-inventory-and-dependencies)

1. Title Page

****Project Name:**** HUEY_P_ClaudeCentric Trading System [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

****Version:**** 1.0.0 [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md] [from: HUEY_P_ClaudeCentric Trading System.txt]

****Release Date:**** June 2025 [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

****Document Status:**** Final [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

****Prepared by:**** Trading System Development Team [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

****Approved by:**** [Approval Authority] [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

2. Executive Summary / Overview

The HUEY_P_ClaudeCentric Trading System is a sophisticated, multi-language automated trading platform designed for the MetaTrader 4 (MT4) environment [from: HUEY_P_ClaudeCentric Trading System.txt]. It is a multi-currency automated trading platform that integrates MetaTrader 4 (MT4) with an intelligent Python-based backend [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]. The system manages

30 currency pairs simultaneously through individual Expert Advisors (EAs), employing machine learning-driven signal generation, hierarchical communication bridges, and advanced reentry optimization logic [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]. The system leverages an intelligent Python backend for ML-driven signal generation and risk management, connected to MQL4 Expert Advisors via a high-performance C++ communication bridge [from: HUEY_P_ClaudeCentric Trading System.txt]. The system is engineered for production use with real capital, featuring a hierarchical communication protocol for resilience, comprehensive configuration management, and robust operational procedures [from: HUEY_P_ClaudeCentric Trading System.txt].

This document provides a high-level overview of the HUEY_P_ClaudeCentric Trading System's architecture [from: HUEY_P_ClaudeCentric Trading System.txt]. It is intended to give all stakeholders a foundational understanding of the system's components, their responsibilities, and how they interact to achieve automated trading [from: HUEY_P_ClaudeCentric Trading System.txt]. This document is the single source of truth for all data formats, schemas, and interfaces that connect the various components of the trading system [from: HUEY_P_ClaudeCentric Trading System.txt].

Key Features:

- **30 Concurrent Trading Pairs:** Individual EAs for each major, cross, and metal pair [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **ML-Powered Signal Generation:** Ensemble models with explainable AI [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Hierarchical Communication:** Three-tier fallback system ensuring 99.9% uptime [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Smart Reentry Logic:** Adaptive position management based on close results [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Real-time Analytics:** Performance tracking, correlation analysis, and risk monitoring [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Automated Deployment:** PowerShell-based orchestration for system-wide updates [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Target Users:

- Professional traders requiring institutional-grade automation [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Trading firms managing multiple currency exposures [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Quantitative analysts implementing complex strategies [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

3. Goals and Objectives

Primary Goals:

- **Automated Multi-Currency Trading:** Execute trades across 30 pairs with minimal manual intervention [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Intelligent Signal Generation:** Leverage ML models for high-confidence trade signals [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Risk Management:** Implement portfolio-level risk controls and position sizing [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **System Reliability:** Achieve 99.9% uptime through redundant communication paths [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Performance Optimization:** Minimize latency while maximizing signal quality [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Measurable Objectives:

- **Signal Latency:** < 100ms from generation to EA receipt [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **System Uptime:** > 99.9% availability [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Trade Execution:** < 50ms from signal to order placement [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Error Rate:** < 0.1% failed communications [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Scalability:** Support up to 50 concurrent pairs without degradation [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Success Criteria:

- All 30 EAs operating simultaneously without conflicts [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Profitable backtesting results across all pairs [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Successful failover between communication bridges [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Real-time monitoring dashboard operational [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Automated deployment completing within 10 minutes [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

4. Background and Context

System Evolution:

The HUEY_P_ClaudeCentric system represents the next generation of the Claude-Enhanced Trading System, addressing previous limitations [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]:

- **Previous Issues:** 30 individual EAs with manual configuration, reactive architecture, limited adaptability [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **New Approach:** Centralized intelligence with distributed execution, proactive risk management, ML-driven adaptation [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Technical Context:

- **MT4 Limitations:** Single-threaded EAs, limited computational power, restricted API [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Bridge Solution:** C++ DLL bridge overcomes MT4 limitations by offloading computation to Python [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **ML Integration:** Python backend enables advanced analytics impossible in MQL4 [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Business Context:

- **Market Requirements:** 24/5 forex markets demand high reliability [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Regulatory Compliance:** System logs all activities for audit trails [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Risk Management:** Portfolio-level controls prevent excessive exposure [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

5. Definitions and Acronyms

Technical Terms:

- **EA (Expert Advisor):** Automated trading program running in MT4 [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md] [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Bridge:** Communication layer between MT4 and Python [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Signal:** Trading instruction with direction, confidence, and parameters [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Reentry:** Logic for re-entering positions after initial close [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Parameter Set:** Configuration defining trade size, stops, and risk [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

System-Specific Terms:

- **HUEY_P:** Project codename (Highly Unified Exchange Yielding Profits) [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

- **ClaudeCentric:** AI-driven decision-making at system core [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **RCI System:** Regional-Country-Impact strategy identification [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Strategy ID:** 5-digit code identifying signal generation strategy [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

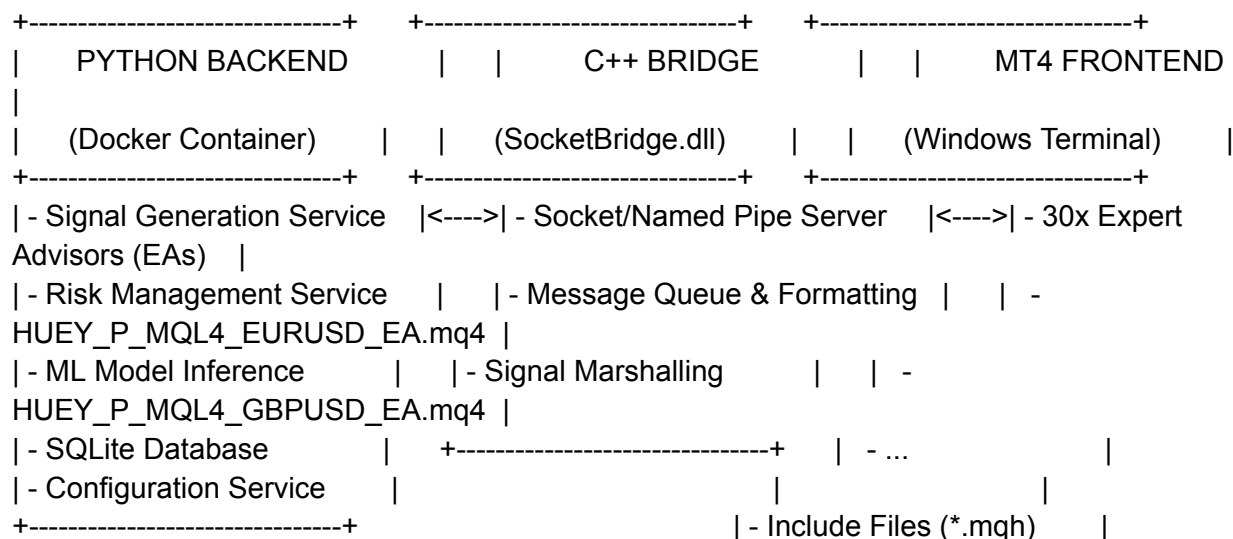
Acronyms:

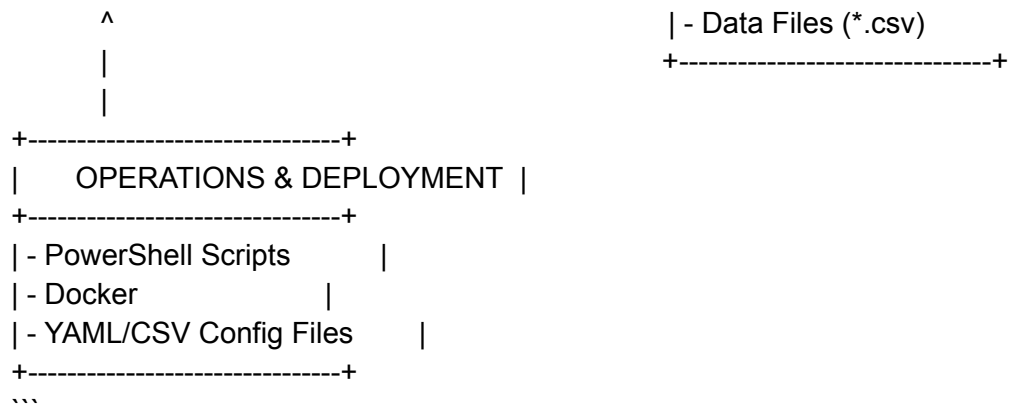
- **DLL:** Dynamic Link Library [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **CSV:** Comma-Separated Values [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **YAML:** Yet Another Markup Language [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **ML:** Machine Learning [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **SL:** Stop Loss [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **TP:** Take Profit [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **UUID:** Universally Unique Identifier [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

6. System Architecture

6.1 High-Level Architecture Diagram

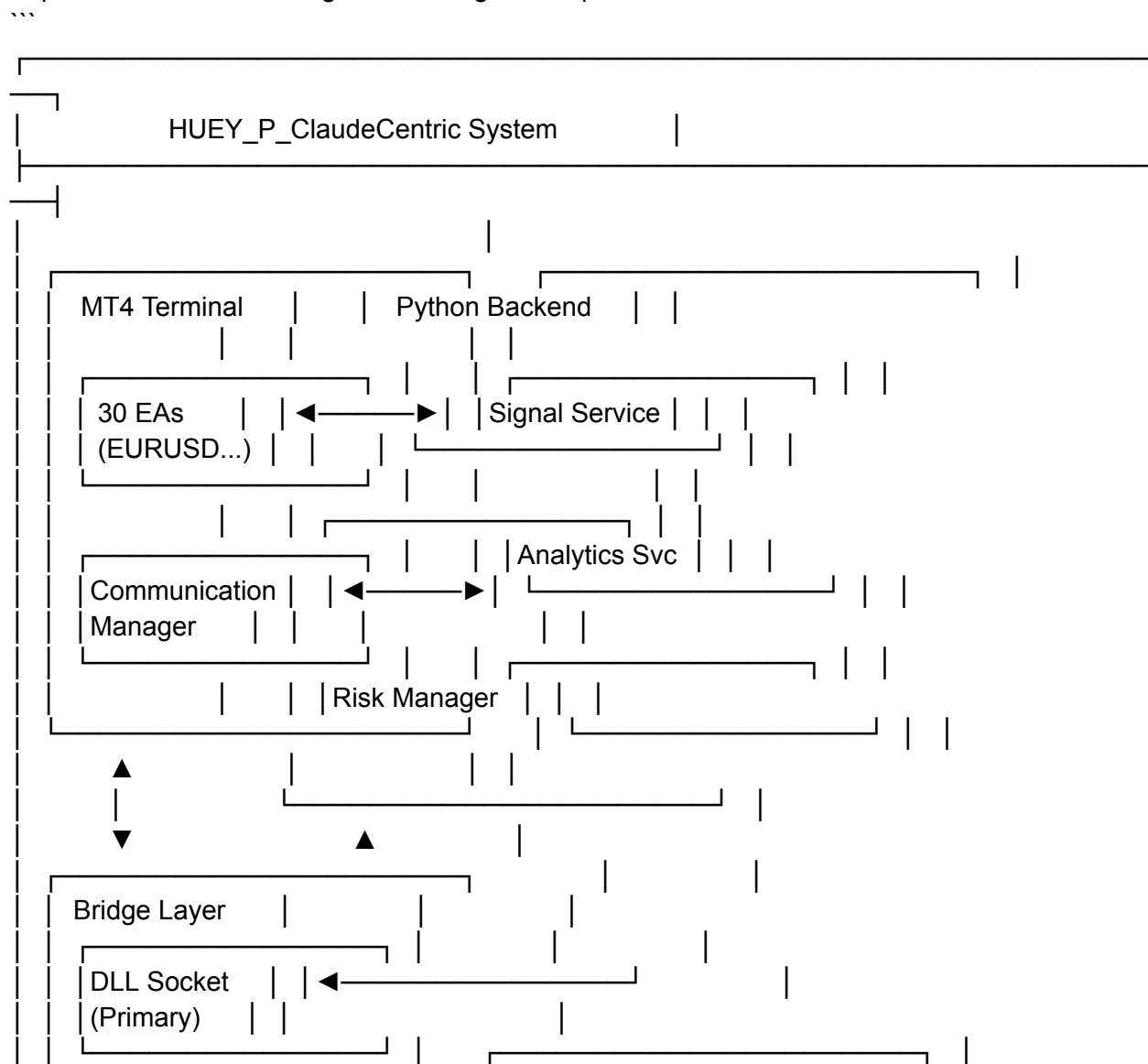
...

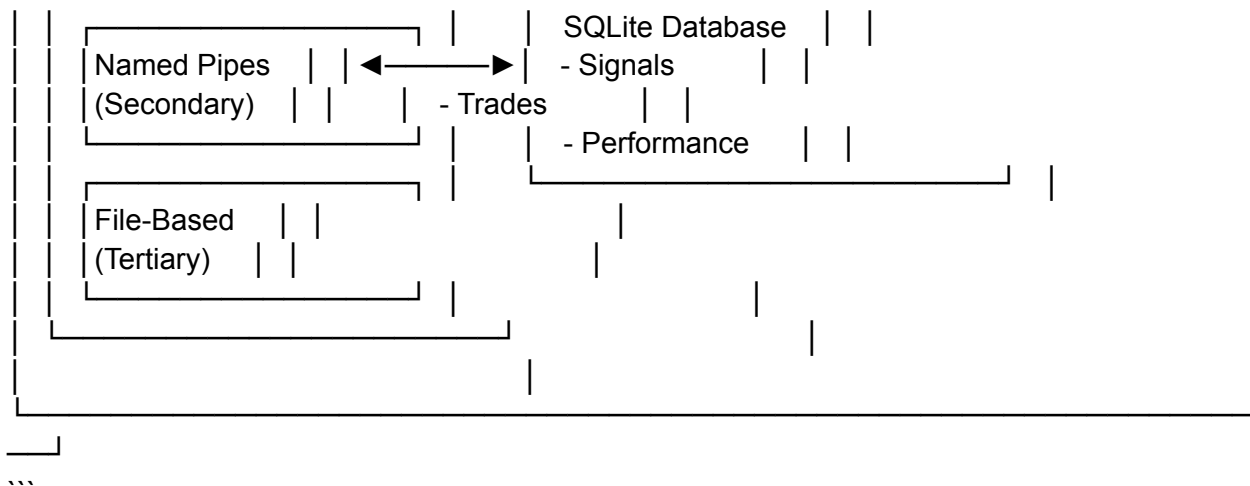




[from: HUEY_P_ClaudeCentric Trading System.txt]

Expanded architecture diagram showing all components:





[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

6.2 Core Architectural Principles

The system is built upon three core principles [from: HUEY_P_ClaudeCentric Trading System.txt]:

1. **Decoupling & Modularity:** Each primary component (MQL4, C++, Python) operates independently [from: HUEY_P_ClaudeCentric Trading System.txt]. This allows for modular updates and maintenance without causing system-wide failures [from: HUEY_P_ClaudeCentric Trading System.txt]. For example, the Python signal generation logic can be updated without recompiling the MQL4 EAs [from: HUEY_P_ClaudeCentric Trading System.txt].
2. **Redundancy & Resilience:** The communication system is designed with a hierarchical fallback mechanism (Socket → Named Pipes → Files) [from: HUEY_P_ClaudeCentric Trading System.txt]. This ensures that trading signals can be delivered even if the primary communication channel fails, a critical feature for a live trading environment [from: HUEY_P_ClaudeCentric Trading System.txt].
3. **Flexibility & Control:** The system is heavily parameter-driven through YAML and CSV configuration files [from: HUEY_P_ClaudeCentric Trading System.txt]. This allows operators to change trading strategies, risk parameters, and system behavior without altering the source code [from: HUEY_P_ClaudeCentric Trading System.txt].

6.3 Technology Stack

- **Frontend:** MT4 Terminal (MQL4) [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Backend:** Python 3.9+ [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md] (Asyncio, Pandas, Scikit-learn) [from: HUEY_P_ClaudeCentric Trading System.txt]

- **Database:** SQLite 3.x [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Cache:** Redis 6.x [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Communication:** C++ DLL [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md] (Windows Sockets API) [from: HUEY_P_ClaudeCentric Trading System.txt], Windows Named Pipes, CSV files [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Deployment:** PowerShell 5.1+ [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md] & Docker [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Monitoring:** Custom Python dashboard [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **Configuration:** YAML & CSV [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Operational Scripts:** PowerShell [from: HUEY_P_ClaudeCentric Trading System.txt]

6.4 Component Descriptions

- **Python Backend:** The "brain" of the system [from: HUEY_P_ClaudeCentric Trading System.txt]. It runs as a set of containerized services responsible for market analysis, machine learning model inference, signal generation, and pre-trade risk validation [from: HUEY_P_ClaudeCentric Trading System.txt].
- **C++ Bridge (SocketBridge.dll):** A high-performance, low-level Windows DLL that acts as the communication intermediary [from: HUEY_P_ClaudeCentric Trading System.txt]. Its sole purpose is to efficiently and reliably transfer messages between the Python backend and the MQL4 EAs [from: HUEY_P_ClaudeCentric Trading System.txt].
- **MT4 Frontend:** The MetaTrader 4 terminal hosts 30 individual Expert Advisors (EAs), one for each traded currency pair [from: HUEY_P_ClaudeCentric Trading System.txt]. The EAs are lightweight, receiving signals from the bridge and executing trades based on parameters defined in shared `.mqh` and `.csv` files [from: HUEY_P_ClaudeCentric Trading System.txt].
- **Operations & Deployment:** A collection of PowerShell scripts, Docker files, and configuration files that manage the entire system lifecycle, from initial setup and deployment to monitoring and maintenance [from: HUEY_P_ClaudeCentric Trading System.txt].

6.5 Python Backend Layer

The Python Backend is the intelligent core of the trading system [from: HUEY_P_ClaudeCentric Trading System.txt]. It operates as a suite of asynchronous services within a Docker container [from: HUEY_P_ClaudeCentric Trading System.txt]. Its responsibilities include market data analysis, signal generation via machine learning models, configuration management, and robust risk assessment [from: HUEY_P_ClaudeCentric Trading System.txt]. The backend is built on

Python's `asyncio` library, enabling high-performance, concurrent I/O operations [from: HUEY_P_ClaudeCentric Trading System.txt].

6.5.1 Core Services

****MarketDataService:****

- ****Purpose:**** Connects to external market data APIs (e.g., a broker's FIX/API or a third-party provider) to stream real-time and historical price data [from: HUEY_P_ClaudeCentric Trading System.txt]

- ****Functionality:**** Normalizes data from different sources into a consistent internal format and distributes it to other services [from: HUEY_P_ClaudeCentric Trading System.txt]

****SignalService (`signal\_generator.py`):****

- ****Purpose:**** The primary signal generation engine [from: HUEY_P_ClaudeCentric Trading System.txt]

- ****Functionality:**** Consumes market data, preprocesses it into features, and feeds these features into the ML models [from: HUEY_P_ClaudeCentric Trading System.txt]. When a model predicts a high-probability trading opportunity, this service constructs a formal signal message [from: HUEY_P_ClaudeCentric Trading System.txt]

****RiskService (`risk\_manager.py`):****

- ****Purpose:**** Acts as a centralized pre-trade gatekeeper to enforce risk management rules [from: HUEY_P_ClaudeCentric Trading System.txt]

- ****Functionality:**** Before a signal is sent to the C++ bridge, it is validated by this service [from: HUEY_P_ClaudeCentric Trading System.txt]. Checks include:

- Maximum drawdown limits [from: HUEY_P_ClaudeCentric Trading System.txt]

- Maximum concurrent open positions [from: HUEY_P_ClaudeCentric Trading System.txt]

- Correlation checks to avoid over-exposure to a single currency [from: HUEY_P_ClaudeCentric Trading System.txt]

- Calculates the appropriate lot size based on `riskPercent` and account equity [from: HUEY_P_ClaudeCentric Trading System.txt]

****BridgeService (`trading\_server.py`):****

- ****Purpose:**** Manages the TCP/IP socket connection to the `SocketBridge.dll` [from: HUEY_P_ClaudeCentric Trading System.txt]

- ****Functionality:**** It takes validated signals from the `RiskService`, formats them according to the API contract, and transmits them to the C++ bridge [from: HUEY_P_ClaudeCentric Trading System.txt]. It also handles connection logic, heartbeating, and retries [from: HUEY_P_ClaudeCentric Trading System.txt]

****ConfigurationService:****

- ****Purpose:**** Loads and provides system-wide access to all YAML configuration files [from: HUEY_P_ClaudeCentric Trading System.txt]

- **Functionality:** It parses the hierarchical configuration and exposes a simple interface for other services to query parameters [from: HUEY_P_ClaudeCentric Trading System.txt]. It can also watch for file changes to enable hot-reloading of certain settings [from: HUEY_P_ClaudeCentric Trading System.txt]

6.5.2 Machine Learning (ML) Components

- **FeatureEngine.py:** A collection of functions for transforming raw market data (prices, volumes) into meaningful features for the models (e.g., rolling averages, RSI, volatility metrics) [from: HUEY_P_ClaudeCentric Trading System.txt]

- **ModelTrainer.py:** An offline script used to train, evaluate, and serialize the ML models. This is not part of the live runtime [from: HUEY_P_ClaudeCentric Trading System.txt]

- **models/ directory:** Contains the serialized, pre-trained model files (e.g., using `joblib` or `pickle`) [from: HUEY_P_ClaudeCentric Trading System.txt]. The `SignalService` loads these models on startup for inference [from: HUEY_P_ClaudeCentric Trading System.txt]. The models might include:

- A classifier to predict market direction (Buy/Sell/Hold) [from: HUEY_P_ClaudeCentric Trading System.txt]
- A regressor to predict the confidence or potential magnitude of a move [from: HUEY_P_ClaudeCentric Trading System.txt]

6.5.3 Python Dependencies

All required Python packages are listed in `requirements.txt` [from: HUEY_P_ClaudeCentric Trading System.txt]. Key dependencies include:

- **asyncio:** Core concurrency framework [from: HUEY_P_ClaudeCentric Trading System.txt]
- **pandas:** Data manipulation and analysis [from: HUEY_P_ClaudeCentric Trading System.txt]
- **numpy:** Numerical operations [from: HUEY_P_ClaudeCentric Trading System.txt]
- **scikit-learn:** Machine learning model inference [from: HUEY_P_ClaudeCentric Trading System.txt]
- **PyYAML:** For parsing YAML configuration files [from: HUEY_P_ClaudeCentric Trading System.txt]
- **sqlalchemy:** For database interaction (Core or ORM) [from: HUEY_P_ClaudeCentric Trading System.txt]

6.6 Bridge Layer

The C++ Bridge, compiled as `SocketBridge.dll`, is a crucial middleware component that enables high-speed, reliable communication between the Python backend and the MQL4 frontend [from: HUEY_P_ClaudeCentric Trading System.txt]. It is written in C++ using the native Windows API for maximum performance and minimum latency [from: HUEY_P_ClaudeCentric Trading System.txt]. The DLL is multi-threaded to ensure non-blocking operations [from: HUEY_P_ClaudeCentric Trading System.txt].

The primary responsibility of the DLL is to act as a **protocol and language translator** [from: HUEY_P_ClaudeCentric Trading System.txt]:

- It listens for incoming TCP/IP socket connections from the Python backend [from: HUEY_P_ClaudeCentric Trading System.txt]
- It receives signal messages, validates their basic format, and places them into a thread-safe queue [from: HUEY_P_ClaudeCentric Trading System.txt]
- It exposes a simple, clean C-style interface for the MQL4 environment to poll for these messages [from: HUEY_P_ClaudeCentric Trading System.txt]
- This decouples the Python and MQL4 environments, so neither needs to know the implementation details of the other [from: HUEY_P_ClaudeCentric Trading System.txt]

Bridge Architecture:

- **Listener Thread:** A dedicated thread is spawned when `StartServer()` is called. This thread's sole responsibility is to listen for and accept incoming socket connections from the Python backend [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Worker Thread(s):** Upon connection, a new thread may be created to handle receiving data from that specific client. This thread reads incoming signal messages, performs basic validation, and pushes them into the central message queue [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Message Queue:** A thread-safe, FIFO (First-In, First-Out) queue (e.g., `std::queue` protected by a `std::mutex`). This queue acts as the buffer between the network-facing threads and the MQL4-facing functions [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Main DLL Thread:** The MQL4 calls (`GetNextSignal`) are handled by the main thread, which simply locks the queue, dequeues a message if available, and returns it [from: HUEY_P_ClaudeCentric Trading System.txt]

Three-Tier Communication Architecture:

The system implements a three-tier communication fallback: DLL Socket → Named Pipes → File-based [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md].

1. **DLL Socket Bridge (Primary):** High-speed communication (< 10ms) [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
2. **Named Pipes (Secondary):** Secondary fallback (< 50ms) [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
3. **File-Based (Tertiary):** Tertiary fallback with batching (< 500ms) [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

6.7 MT4 Terminal Layer

The MQL4 Subsystem consists of all components operating within the MetaTrader 4 terminal [from: HUEY_P_ClaudeCentric Trading System.txt]. Its primary role is trade execution and management [from: HUEY_P_ClaudeCentric Trading System.txt]. It acts as the "hands" of the

trading system, receiving high-level commands from the Python backend and translating them into trade orders [from: HUEY_P_ClaudeCentric Trading System.txt].

6.7.1 Expert Advisors (EAs)

The system employs 30 individual EAs, one for each currency pair (e.g., `HUEY\_P\_MQL4\_EURUSD\_EA.mq4`) [from: HUEY_P_ClaudeCentric Trading System.txt]. This design isolates risk and simplifies debugging [from: HUEY_P_ClaudeCentric Trading System.txt].

- **Architecture:** While there are 30 `.mq4` files, they all contain nearly identical boilerplate code. The core trading logic is abstracted into the shared `.mqh` include files [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Purpose:** Each EA is responsible for the trading activity of a single currency pair. It initializes communication with the bridge, polls for signals relevant to its pair, executes trades, and manages open positions for that pair only [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Configuration:** Key EA parameters like `MagicNumber`, `MaxSpread`, and file paths are configured via the `Inputs` tab when the EA is attached to a chart [from: HUEY_P_ClaudeCentric Trading System.txt]

6.7.2 Include Files (`/MQL4/Include`)

These files contain the shared logic used by all EAs, promoting code reuse and consistency [from: HUEY_P_ClaudeCentric Trading System.txt]:

- **CommunicationManager.mqh:** Manages all interaction with the `SocketBridge.dll`. It handles importing DLL functions, establishing connections, and retrieving signals [from: HUEY_P_ClaudeCentric Trading System.txt]
- **TradingCore.mqh:** The heart of the trading logic. It receives a processed signal and a parameter set, then performs the necessary pre-flight checks (e.g., verifying spread) before passing the trade to the execution engine [from: HUEY_P_ClaudeCentric Trading System.txt]
- **ExecutionEngine.mqh:** A low-level module responsible for the `OrderSend()` function. It includes logic for retrying failed orders due to requotes or other transient broker errors [from: HUEY_P_ClaudeCentric Trading System.txt]
- **FileManager.mqh:** Provides functions for loading and parsing the various `.csv` configuration files from the `/MQL4/Files` directory [from: HUEY_P_ClaudeCentric Trading System.txt]
- **TimeManager.mqh:** Handles time-based trading rules, such as preventing trades around major news events by reading a `NewsCalendar.csv` [from: HUEY_P_ClaudeCentric Trading System.txt]
- **ErrorRecovery.mqh:** A state machine for handling system errors, such as a prolonged loss of communication with the backend [from: HUEY_P_ClaudeCentric Trading System.txt]

- **Logging.mqh**: Provides standardized logging functions for writing messages to the Experts log file [from: HUEY_P_ClaudeCentric Trading System.txt]

6.7.3 MQL4 Execution Engine (`ExecutionEngine.mqh`)

The MQL4 Execution Engine (`ExecutionEngine.mqh`) serves as the critical "hands" of the HUEY_P_ClaudeCentric Trading System, responsible for the low-level interaction with the MetaTrader 4 terminal [from: MQL4 Execution Engine Technical Documentation.md]. As documented in `HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md` and `HUEY\_P\_ClaudeCentric Trading System.txt`, this component acts as the final executor of trading decisions made by the Python backend's ML-driven signal generation service [from: MQL4 Execution Engine Technical Documentation.md].

The Execution Engine's primary purpose is to [from: MQL4 Execution Engine Technical Documentation.md]:

- Translate high-level trading signals into actual MT4 `OrderSend()` commands [from: MQL4 Execution Engine Technical Documentation.md]
- Handle transient broker errors with intelligent retry mechanisms [from: MQL4 Execution Engine Technical Documentation.md]
- Ensure reliable trade execution across 30 concurrent currency pairs [from: MQL4 Execution Engine Technical Documentation.md]
- Report execution status back through the communication bridge [from: MQL4 Execution Engine Technical Documentation.md]

The Execution Engine operates within a carefully orchestrated sequence of MQL4 functions [from: MQL4 Execution Engine Technical Documentation.md]:

1. **Signal Reception**: The `CommunicationManager.mqh` continuously polls for signals using `GetNextSignal()` from the `SocketBridge.dll` [from: MQL4 Execution Engine Technical Documentation.md]
2. **Signal Processing**: Valid signals are passed to `TradingCore.mqh` via `ProcessSignal()` [from: MQL4 Execution Engine Technical Documentation.md]
3. **Trade Execution**: `TradingCore` invokes `ExecutionEngine.ExecuteTrade()` with prepared parameters [from: MQL4 Execution Engine Technical Documentation.md]
4. **Error Handling**: The Execution Engine implements retry logic for failed trades [from: MQL4 Execution Engine Technical Documentation.md]
5. **Status Reporting**: Results are communicated back through the bridge [from: MQL4 Execution Engine Technical Documentation.md]

The engine employs a sophisticated retry mechanism specifically designed to handle transient broker errors [from: MQL4 Execution Engine Technical Documentation.md]:

- **Error 130**: Invalid stops (requotes or spread changes) [from: MQL4 Execution Engine Technical Documentation.md]

- **Error 136**: No prices (market volatility or connectivity issues) [from: MQL4 Execution Engine Technical Documentation.md]
- **Error 138**: Requote situations [from: MQL4 Execution Engine Technical Documentation.md]

The retry logic implements exponential backoff to avoid overwhelming the broker server while maximizing the chance of successful execution [from: MQL4 Execution Engine Technical Documentation.md].

The Execution Engine is designed for minimal latency and maximum reliability [from: MQL4 Execution Engine Technical Documentation.md]:

- **Pre-execution Validation**: < 1ms [from: MQL4 Execution Engine Technical Documentation.md]
- **Order Preparation**: < 2ms [from: MQL4 Execution Engine Technical Documentation.md]
- **MT4 OrderSend() Call**: 20-50ms (broker-dependent) [from: MQL4 Execution Engine Technical Documentation.md]
- **Status Reporting**: < 5ms [from: MQL4 Execution Engine Technical Documentation.md]
- **Total execution engine overhead**: < 10ms (excluding broker latency) [from: MQL4 Execution Engine Technical Documentation.md]

6.7.4 Data Files (`/MQL4/Files/`)

These CSV files provide runtime configuration, allowing trading behavior to be modified without recompiling the EAs [from: HUEY_P_ClaudeCentric Trading System.txt]:

- **all_10_parameter_sets.csv**: Defines multiple sets of trading parameters (e.g., Stop Loss, Take Profit, Risk Percentage) [from: HUEY_P_ClaudeCentric Trading System.txt]
- **signal_id_mapping.csv**: Maps an incoming signal's `strategy\_id` to a specific parameter set from the file above [from: HUEY_P_ClaudeCentric Trading System.txt]
- **reentry_close_result_mapping.csv**: Contains rules for adjusting behavior based on the result of the last trade (the "DIE 1-6" strategy) [from: HUEY_P_ClaudeCentric Trading System.txt]
- **current_signal.csv**: The fallback communication file, used only if Socket and Named Pipe connections fail [from: HUEY_P_ClaudeCentric Trading System.txt]

6.8 Data Layer

SQLite Database:

- **Technology**: SQLite is used for its simplicity and file-based nature, which is ideal for a self-contained containerized application [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Purpose**: [from: HUEY_P_ClaudeCentric Trading System.txt]
 - To log every signal generated [from: HUEY_P_ClaudeCentric Trading System.txt]
 - To record every trade execution confirmation received from the frontend [from: HUEY_P_ClaudeCentric Trading System.txt]

- To store historical performance metrics for analysis [from: HUEY_P_ClaudeCentric Trading System.txt]
- **Schema:** The database schema defines tables for `signals`, `trades`, and `system\_logs`. See the API Contract for the full DDL [from: HUEY_P_ClaudeCentric Trading System.txt]

Redis Cache: Used for high-speed data caching for real-time operations [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

6.9 PowerShell Management Layer

PowerShell Scripts (`/Scripts`): A collection of PowerShell scripts for automation, deployment, and monitoring [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

6.10 Excel Dashboard Layer

Excel Dashboard (`.xlsm`): A real-time trading dashboard with manual override capabilities [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

6.11 External Dependencies

- **MT4 Terminal:** The MetaTrader 4 trading platform [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- **External Market Feed:** External market data provider [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

7. System Behavior and Workflows

7.1 Data Flow Diagram: Signal to Execution

This diagram illustrates the lifecycle of a single trading signal [from: HUEY_P_ClaudeCentric Trading System.txt]:

...

(START)

|
V

[Python: Market Data Analysis] --> [Python: ML Model Predicts Opportunity]

|
V

[Python: Signal Generated (e.g., BUY EURUSD)]

|

```

V
[Python: Risk Manager Validates Trade (Lot size, exposure)]
|
V
[Python: Signal formatted and sent to C++ Bridge via Socket]
|
V
[C++ Bridge: Receives signal, places in queue]
|
V
[MQL4 EA: Polls C++ Bridge for new signal via GetNextSignal()]
|
V
[MQL4 EA: Receives signal string for its pair (EURUSD)]
|
V
[MQL4 EA: Parses signal, loads parameters from *.csv files]
|
V
[MQL4 EA: Performs final checks (spread, slippage)]
|
V
[MQL4 EA: Executes OrderSend() trade command]
|
V
(END)
...

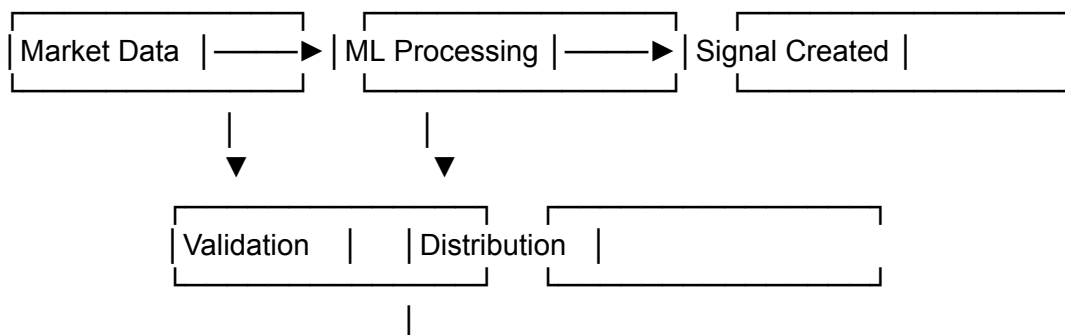
```

[from: HUEY_P_ClaudeCentric Trading System.txt]

7.2 Signal Generation Workflow

Detailed workflow from market data to signal distribution [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]:

...



▼
EA Execution |

...

[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Detailed Steps [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]:

1. **Market Data Collection (Every tick)**

- Collect bid/ask prices [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Calculate technical indicators [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Check economic calendar [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

2. **ML Processing (Triggered by events)**

- Run ensemble models [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Generate confidence scores [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Apply strategy rules [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

3. **Signal Validation**

- Check risk limits [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Verify market conditions [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Confirm pair eligibility [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

4. **Signal Distribution**

- Select communication bridge [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Send to target EA [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Await acknowledgment [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

5. **EA Execution**

- Receive signal [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

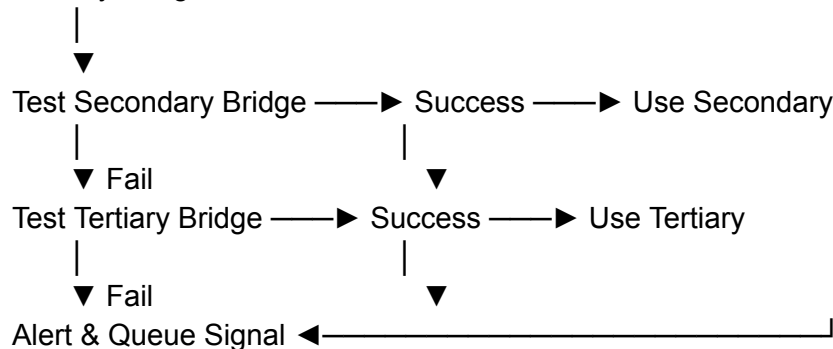
- Apply parameter set [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]
- Execute trade [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

7.3 Error Handling Workflow

****Bridge Failure Scenario**** [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]:

...

Primary Bridge Fail



...

[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

****Trade Errors:**** The `ExecutionEngine.mqh` handles standard MT4 trade errors (e.g., error 130 - invalid stops, error 136 - no prices) with a retry mechanism [from: HUEY_P_ClaudeCentric Trading System.txt].

****Communication Errors:**** `CommunicationManager.mqh` detects failed connections to the bridge [from: HUEY_P_ClaudeCentric Trading System.txt]. After a set number of failures, `ErrorRecovery.mqh` transitions the system state to `DEGRADED` and attempts to use the fallback communication method [from: HUEY_P_ClaudeCentric Trading System.txt].

****Configuration Errors:**** If CSV files are missing or malformed, the EA will fail during `OnInit()` and log a critical error, preventing it from trading [from: HUEY_P_ClaudeCentric Trading System.txt].

****Signal Processing Error**** [from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]:

```
```python
```

```
try:
```

```
 signal = generate_signal(market_data)
```

```
 validate_signal(signal)
```

```
 distribute_signal(signal)
```

```
except ValidationError as e:
```

```

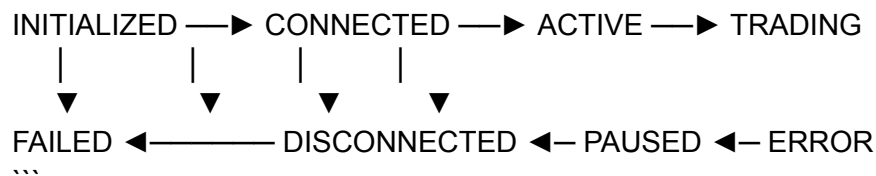
 log_error(f"Signal validation failed: {e}")
 increment_metric("signal_validation_errors")
except DistributionError as e:
 log_error(f"Signal distribution failed: {e}")
 queue_for_retry(signal)
except Exception as e:
 log_critical(f"Unexpected error: {e}")
 alert_admin(e)
...

```

[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

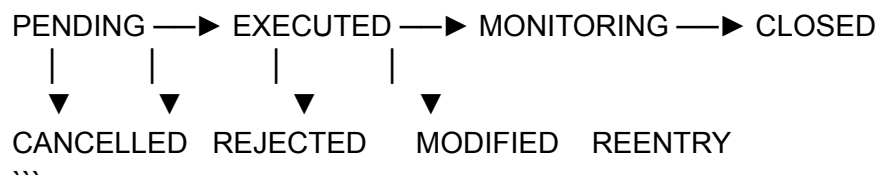
### ### 7.4 State Transitions

**\*\*EA States\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]:



[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

**\*\*Trade States\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]:



[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

---

## ## 8. Functional Requirements

### ### FR-001: Multi-Currency EA Management

- **\*\*ID:\*\*** FR-001 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Priority:\*\*** Critical [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Description:\*\*** System shall deploy and manage 30 individual EAs, one for each supported currency pair [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Use Cases:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-001.1: Deploy EA for single currency pair [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

- UC-001.2: Deploy all 30 EAs simultaneously [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- UC-001.3: Update EA parameters without restart [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Inputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Currency pair symbol (e.g., "EURUSD") [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - EA configuration file path [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Parameter set selection (1-10) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Outputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - EA deployment status [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Initialization confirmation [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Heartbeat signals every 5 seconds [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Error Conditions:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Invalid currency pair [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Missing configuration file [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - MT4 terminal not responding [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### ### FR-002: Signal Generation and Distribution

- **\*\*ID:\*\*** FR-002 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Priority:\*\*** Critical [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Description:\*\*** Python backend shall generate ML-based trading signals and distribute to appropriate EAs [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Use Cases:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-002.1: Generate signal based on market conditions [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-002.2: Route signal to correct EA [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-002.3: Handle signal acknowledgment [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Inputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

- Market data (bid, ask, timestamp) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Technical indicators [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Economic calendar events [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Outputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Signal object with UUID [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Strategy ID (5-digit) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Confidence score (0.0-1.0) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Target currency pair [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Error Conditions:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Insufficient market data [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - ML model failure [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Signal routing timeout [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

### ### FR-003: Hierarchical Communication Bridge

- **\*\*ID:\*\*** FR-003 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Priority:\*\*** Critical [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Description:\*\*** System shall implement three-tier communication fallback: DLL Socket → Named Pipes → File-based [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Use Cases:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-003.1: Primary socket communication [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-003.2: Failover to named pipes [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-003.3: Final fallback to file-based [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Inputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Message payload (JSON/CSV) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

- Target EA identifier [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Priority level [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Outputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Delivery confirmation [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Latency metrics [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Bridge status [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Error Conditions:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Socket connection failure [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Named pipe timeout [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - File system full [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### ### FR-004: Reentry Logic Management

- **\*\*ID:\*\*** FR-004 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Priority:\*\*** High [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Description:\*\*** System shall implement configurable reentry logic based on trade close results [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Use Cases:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-004.1: Analyze close reason [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-004.2: Determine reentry parameters [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-004.3: Execute reentry trade [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Inputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Close result code (1-6) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Original trade parameters [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Market conditions at close [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **\*\*Expected Outputs:\*\*** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Reentry decision [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

- Modified parameters [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Execution status [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Error Conditions:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Invalid close result [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Insufficient margin [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Reentry disabled [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### ### FR-005: Real-time Monitoring Dashboard

- **ID:** FR-005 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Priority:** High [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Description:** System shall provide real-time monitoring of all 30 pairs and system health [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Use Cases:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-005.1: View all pair statuses [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-005.2: Monitor system performance [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - UC-005.3: Generate alerts [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Expected Inputs:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - EA heartbeat data [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Trade execution logs [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - System metrics [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Expected Outputs:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Dashboard display [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Performance graphs [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Alert notifications [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Error Conditions:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

- Dashboard service down [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Data feed interruption [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Display rendering failure [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

---

## ## 9. Non-Functional Requirements

### ### 9.1 Performance Requirements

- **ID:** NFR-001 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Category:** Performance [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Description:** System performance specifications [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Specifications:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Signal generation: < 50ms [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Signal delivery: < 10ms (primary bridge) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Trade execution: < 50ms [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Dashboard refresh: < 1 second [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Database query: < 100ms [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Measurement Method:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Automated performance tests [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Production monitoring metrics [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Load testing scenarios [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

### ### 9.2 Reliability and Availability

- **ID:** NFR-002 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Category:** Reliability [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

- **Description:** System uptime and fault tolerance [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Specifications:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - System availability: 99.9% (< 8.76 hours downtime/year) [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Bridge failover time: < 500ms [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Data persistence: No loss on crash [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Recovery time: < 5 minutes [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Measurement Method:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Uptime monitoring [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Failover testing [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Disaster recovery drills [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

### ### 9.3 Security Requirements

- **ID:** NFR-003 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Category:** Security [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Description:** System security measures [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Specifications:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Communication encryption: TLS 1.3 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Authentication: API key + IP whitelist [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Audit logging: All actions logged [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Access control: Role-based permissions [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Measurement Method:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Security audits [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Penetration testing [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

- Compliance reviews [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### ### 9.4 Scalability Requirements

- **ID:** NFR-004 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Category:** Scalability [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Description:** System growth capabilities [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Specifications:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Currency pairs: Support up to 50 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Concurrent EAs: Handle 50+ simultaneously [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Data storage: 5 years historical data [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Processing: Linear scaling with pairs [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Measurement Method:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Load testing [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Capacity planning [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Performance benchmarks [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### ### 9.5 Usability Requirements

- **ID:** NFR-005 [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Category:** Usability [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Description:** User interface and experience [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Specifications:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Dashboard load time: < 3 seconds [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Configuration changes: No restart required [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
  - Error messages: Clear and actionable [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

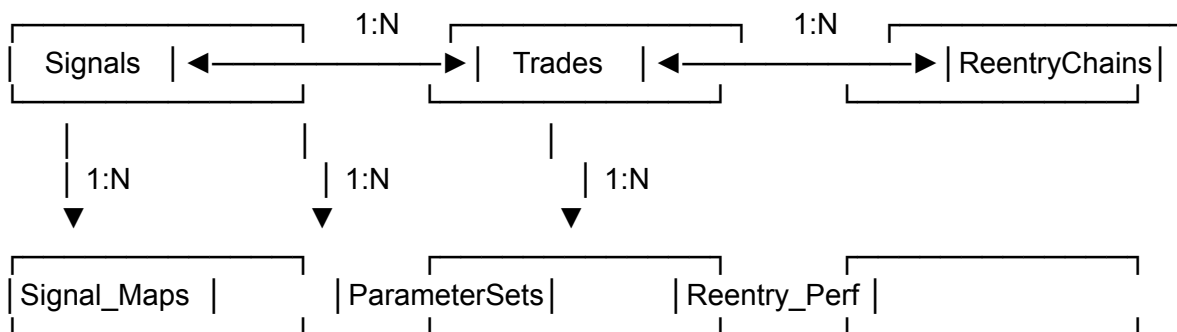
- Documentation: Complete and current [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- **Measurement Method:** [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- User acceptance testing [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Usability studies [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]
- Support ticket analysis [from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

---

## ## 10. Data Models

### ### 10.1 Entity Relationship Diagram

...



...

[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

### ### 10.2 Core Data Models

#### #### 10.2.1 Signal Model (SQL)

```

```sql
CREATE TABLE signals (
  id TEXT PRIMARY KEY,          -- UUID
  symbol TEXT NOT NULL,         -- Currency pair
  direction TEXT CHECK (direction IN ('BUY', 'SELL')),
  confidence REAL CHECK (confidence BETWEEN 0.0 AND 1.0),
  timestamp TIMESTAMP NOT NULL,
  strategy_id INTEGER NOT NULL,  -- 5-digit strategy code
  metadata TEXT,                -- JSON additional data
  processed BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

```
);  
...
```

[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Alternative schema definition:

```
```sql  
-- Table to log every signal generated by the Python backend
CREATE TABLE IF NOT EXISTS signals (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 signal_id TEXT NOT NULL UNIQUE,
 timestamp TEXT NOT NULL,
 symbol TEXT NOT NULL,
 command TEXT NOT NULL,
 confidence REAL NOT NULL,
 strategy_id TEXT NOT NULL,
 sl_points INTEGER,
 tp_points INTEGER,
 sent_to_bridge_at TEXT
);
...
```

[from: HUEY\_P\_ClaudeCentric Trading System.txt]

#### 10.2.2 Trade Model (SQL)

```
```sql  
CREATE TABLE trades (  
  id TEXT PRIMARY KEY,          -- UUID  
  signal_id TEXT NOT NULL,  
  symbol TEXT NOT NULL,  
  direction TEXT,  
  lot_size REAL NOT NULL CHECK (lot_size > 0),  
  entry_price REAL,  
  exit_price REAL,  
  stop_loss REAL,  
  take_profit REAL,  
  status TEXT CHECK (status IN ('PENDING', 'EXECUTED', 'CLOSED', 'CANCELLED',  
'FAILED')),  
  pnl REAL DEFAULT 0.0,  
  opened_at TIMESTAMP,  
  closed_at TIMESTAMP,  
  close_reason INTEGER,         -- Maps to reentry logic  
  mt4_ticket INTEGER,  
  magic_number INTEGER,  
  parameter_set INTEGER,  
  FOREIGN KEY (signal_id) REFERENCES signals (id)
```

```
);  
...
```

[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

Alternative schema definition:

```
```sql  
-- Table to log every trade executed by the MQL4 frontend
CREATE TABLE IF NOT EXISTS trades (
 ticket INTEGER PRIMARY KEY,
 signal_id TEXT,
 symbol TEXT NOT NULL,
 open_time TEXT NOT NULL,
 trade_type TEXT NOT NULL, -- BUY, SELL
 volume REAL NOT NULL,
 open_price REAL NOT NULL,
 sl_price REAL,
 tp_price REAL,
 close_time TEXT,
 close_price REAL,
 profit REAL,
 FOREIGN KEY (signal_id) REFERENCES signals (signal_id)
);
...
```

[from: HUEY\_P\_ClaudeCentric Trading System.txt]

##### 10.2.3 Parameter Set Model

```
```python  
@dataclass  
class ParameterSet:  
    set_id: int          # 1-10  
    lot_size: Decimal  
    stop_loss_pips: int  
    take_profit_pips: int  
    trailing_stop_enabled: bool  
    risk_percentage: Decimal  
    magic_number: int  
    max_spread: int  
...
```

[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

10.2.4 Reentry Configuration Model

```
```python  
@dataclass
class ReentryConfig:
```

```

close_result: int # 1-6
action: str # 'NO_REENTRY', 'IMMEDIATE', 'DELAYED'
size_multiplier: Decimal
delay_seconds: int
confidence_adjustment: Decimal
parameter_overrides: Dict[str, Any]
...

```

[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

---

## ## 11. API Specifications

This document is the **\*\*single source of truth\*\*** for all data formats, schemas, and interfaces that connect the various components of the trading system [from: HUEY\_P\_ClaudeCentric Trading System.txt]. Adherence to these contracts is mandatory to ensure system integrity and interoperability [from: HUEY\_P\_ClaudeCentric Trading System.txt].

### ### 11.1 Communication Bridge API (SocketBridge.dll)

These are the C++ functions exported by the DLL and imported by the MQL4  
`CommunicationManager.mqh` [from: HUEY\_P\_ClaudeCentric Trading System.txt]:

```

```cpp
// Starts the server, binds to a port, and begins listening.
// Returns 0 on success, negative value on error.
__declspec(dllexport) int __stdcall StartServer(int port);

// Signals all threads to stop, closes sockets, and cleans up resources.
__declspec(dllexport) void __stdcall StopServer();

// Retrieves the next available signal from the internal queue.
// The MQL4 side provides a buffer.
// Returns the number of characters copied, or 0 if no signal, or a negative value on error.
__declspec(dllexport) int __stdcall GetNextSignal(wchar_t* buffer, int bufferSize);

// Returns the current status of the bridge as an integer code.
__declspec(dllexport) int __stdcall GetStatus();
...

```

[from: HUEY_P_ClaudeCentric Trading System.txt]

11.2 Signal Message Format (Socket/Named Pipe)

Signals are transmitted as UTF-8 encoded, semicolon-delimited strings [from: HUEY_P_ClaudeCentric Trading System.txt]. The C++ DLL converts this to a UTF-16 wide string for MQL4 [from: HUEY_P_ClaudeCentric Trading System.txt].

****Format:****

`SignalID;Timestamp;Symbol;Command;Confidence;StrategyID;SL\_Points;TP\_Points` [from: HUEY_P_ClaudeCentric Trading System.txt]

****Example:****

`"12345;2025-06-28T10:30:00Z;EURUSD;BUY;0.85;Strategy-A5;150;300"` [from: HUEY_P_ClaudeCentric Trading System.txt]

Field	Type	Description	
`SignalID`	String	Unique identifier for this signal.	[from: HUEY_P_ClaudeCentric Trading System.txt]
`Timestamp`	String	ISO 8601 UTC timestamp of signal generation.	[from: HUEY_P_ClaudeCentric Trading System.txt]
`Symbol`	String	The currency pair, e.g., "EURUSD".	[from: HUEY_P_ClaudeCentric Trading System.txt]
`Command`	String	Trading command: "BUY", "SELL", "CLOSE".	[from: HUEY_P_ClaudeCentric Trading System.txt]
`Confidence`	Float	ML model's confidence in the signal (0.0 to 1.0).	[from: HUEY_P_ClaudeCentric Trading System.txt]
`StrategyID`	String	Identifier for the generating strategy.	[from: HUEY_P_ClaudeCentric Trading System.txt]
`SL_Points`	Integer	Suggested Stop Loss in integer points.	[from: HUEY_P_ClaudeCentric Trading System.txt]
`TP_Points`	Integer	Suggested Take Profit in integer points.	[from: HUEY_P_ClaudeCentric Trading System.txt]

11.3 Error and Status Code Definitions

The DLL functions return integer codes to MQL4 [from: HUEY_P_ClaudeCentric Trading System.txt]:

Code	Constant	Meaning	
1	`STATUS_CONNECTED`	Connected to a client (Python backend).	[from: HUEY_P_ClaudeCentric Trading System.txt]
0	`STATUS_LISTENING`	Server is running and listening, but no client.	[from: HUEY_P_ClaudeCentric Trading System.txt]
-1	`STATUS_STOPPED`	Server is not running.	[from: HUEY_P_ClaudeCentric Trading System.txt]

| -2 | `ERROR\_WINSOCK\_FAIL` | Failed to initialize Windows Sockets. | [from:
HUEY_P_ClaudeCentric Trading System.txt]
| -3 | `ERROR\_SOCKET\_FAIL` | Failed to create the listening socket. | [from:
HUEY_P_ClaudeCentric Trading System.txt]
| -4 | `ERROR\_BIND\_FAIL` | Failed to bind the socket to the specified port. | [from:
HUEY_P_ClaudeCentric Trading System.txt]
| -10 | `ERROR\_NO\_SIGNAL` | Returned by `GetNextSignal` when the queue is empty. |
[from: HUEY_P_ClaudeCentric Trading System.txt]
| -11 | `ERROR\_BUFFER\_TOO\_SMALL` | The MQL4 buffer is too small for the signal message.
| [from: HUEY_P_ClaudeCentric Trading System.txt]

11.4 REST API Endpoints

Signal Generation API

```yaml

endpoint: POST /api/v1/signals/generate

authentication: Bearer Token

request:

headers:

Content-Type: application/json

Authorization: Bearer {api\_key}

body:

symbol: string (required)

timeframe: string (default: "M5")

force\_generation: boolean (default: false)

response:

200:

body:

signal\_id: string (UUID)

symbol: string

direction: string ("BUY" | "SELL")

confidence: number (0.0-1.0)

strategy\_id: integer

timestamp: string (ISO 8601)

metadata: object

400:

body:

error: string

details: object

...

[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### #### Trade Execution Status API

```yaml

endpoint: GET /api/v1/trades/{trade_id}

authentication: Bearer Token

request:

headers:

Authorization: Bearer {api_key}

path_params:

trade_id: string (UUID)

response:

200:

body:

trade_id: string

signal_id: string

status: string

entry_price: number

current_price: number

pnl: number

opened_at: string (ISO 8601)

404:

body:

error: "Trade not found"

...

[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

System Health API

```yaml

endpoint: GET /api/v1/health

authentication: None (public)

response:

200:

body:

status: string ("healthy" | "degraded" | "unhealthy")

components:

database: object

redis: object

bridges: array

eas: array

uptime\_seconds: integer

last\_signal: string (ISO 8601)

...

[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### ### 11.5 WebSocket API

##### #### Real-time Signal Stream

```

```javascript
// Connection
ws://localhost:8889/ws/signals

// Subscribe message
{
  "action": "subscribe",
  "channels": ["signals", "trades"],
  "pairs": ["EURUSD", "GBPUSD"]
}

```

```

// Signal event
{
  "type": "signal",
  "data": {
    "signal_id": "uuid",
    "symbol": "EURUSD",
    "direction": "BUY",
    "confidence": 0.85,
    "timestamp": "2025-06-27T10:00:00Z"
  }
}
...

```

[from: HUEY_P_ClaudeCentric Trading System Technical Specification.02.md]

11.6 Internal Communication Protocol

DLL Socket Protocol

```

```cpp
// Message Format
struct SocketMessage {
 char message_id[37]; // UUID string
 char message_type[20]; // "SIGNAL", "HEARTBEAT", "ACK"
 char payload[1024]; // JSON payload
 int priority; // 1-5 (1 highest)
 long timestamp; // Unix timestamp
};

// Example Signal Message
{
 "message_id": "550e8400-e29b-41d4-a716-446655440000",
 "message_type": "SIGNAL",
 "payload": "{\"symbol\":\"EURUSD\",\"direction\":\"BUY\",\"confidence\":0.85}",
 "priority": 1,

```

```
"timestamp": 1719486000
}
...
```

[from: HUEY\_P\_ClaudeCentric Trading System Technical Specification.02.md]

#### #### Socket Protocol Details

- **Transport:** TCP/IP [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Role:** Primary communication channel [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Behavior:** The Python backend acts as the client and initiates the connection to the C++ DLL server. If the connection is lost, the client will attempt to reconnect every 5 seconds for 3 minutes before failing over [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Message Framing:** Messages are delimited by a newline character (`\n`) [from: HUEY\_P\_ClaudeCentric Trading System.txt]

#### #### Named Pipe Specifications

- **Role:** Secondary (fallback) communication channel [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Pipe Name:** `\\.\pipe\HUEY\_P\_ClaudeCentric\_SignalPipe` [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Behavior:** If a socket connection cannot be established, the system will attempt to use a Windows Named Pipe. The message format is identical to the socket protocol [from: HUEY\_P\_ClaudeCentric Trading System.txt]

#### #### File-Based Communication

- **Role:** Tertiary (last resort) communication channel [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **File:** `/MQL4/Files/current\_signal.csv` [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Behavior:** [from: HUEY\_P\_ClaudeCentric Trading System.txt]
  1. The Python backend writes the signal to a temporary file, e.g., `current\_signal.tmp` [from: HUEY\_P\_ClaudeCentric Trading System.txt]
  2. Once the write is complete, it performs an atomic rename of `current\_signal.tmp` to `current\_signal.csv` [from: HUEY\_P\_ClaudeCentric Trading System.txt]
  3. The MQL4 EA periodically checks the modification time of `current\_signal.csv`. If it has changed, it opens the file, reads the signal, and then clears the file content to signify it has been processed. This prevents reprocessing the same signal [from: HUEY\_P\_ClaudeCentric Trading System.txt]

#### #### CSV File Schemas

- **all\_10\_parameter\_sets.csv** [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Purpose:** Defines reusable sets of trading parameters [from: HUEY\_P\_ClaudeCentric Trading System.txt]
- **Schema:** [from: HUEY\_P\_ClaudeCentric Trading System.txt]

ColumnName	DataType	Description	Example	
id	string	Unique identifier for the parameter set.	"aggressive"	[from: HUEY_P_ClaudeCentric Trading System.txt]
stopLoss	int	Stop loss in points.	200	[from: HUEY_P_ClaudeCentric Trading System.txt]
takeProfit	int	Take profit in points.	400	[from: HUEY_P_ClaudeCentric Trading System.txt]
trailingStop	int	Trailing stop activation points.	50	[from: HUEY_P_ClaudeCentric Trading System.txt]
riskPercent	double	Risk per trade as a percentage of equity.	1.5	[from: HUEY_P_ClaudeCentric Trading System.txt]
maxPositions	int	Max concurrent positions for this strategy.	2	[from: HUEY_P_ClaudeCentric Trading System.txt]
useTrailing	bool	`true` to enable trailing stop.	true	[from: HUEY_P_ClaudeCentric Trading System.txt]
description	string	Human-readable description.	"High risk"	[from: HUEY_P_ClaudeCentric Trading System.txt]

\*\*